

Principal Dev Framework

Spec-Kit × GStack — Concise Reference

Spec-Kit owns **definition** (what to build, why, how). GStack owns **delivery** (review, test, ship, reflect). Bridge commands connect them. Every specialist output gets a persistent file — not chat residue.

Cycle at a Glance

DEFINE (Spec-Kit)	CHALLENGE (GStack)	BUILD + SHIP (Both)
/speckit.constitution /speckit.specify /speckit.plan /speckit.tasks /speckit.analyze	/plan-ceo-review /plan-eng-review /plan-design-review (if UI)	Per task in tasks.md: implement /review /qa /cso (if auth/data) /ship /retro (weekly)
└── /speckit.gstack-handoff ──┘		

Exit gates: Spec locked → Eng Review CLEARED → Task shipped with all delivery gates green.

Artifact Map

All artifacts live in repo. Nothing important stays in chat.

Artifact	Location	Written By
Constitution	<code>.specify/memory/constitution.md</code>	<code>/speckit.constitution</code>
Spec	<code>.specify/specs/<branch>/spec.md</code>	<code>/speckit.specify</code>
Plan	<code>.specify/specs/<branch>/plan.md</code>	<code>/speckit.plan</code>
Tasks	<code>.specify/specs/<branch>/tasks.md</code>	<code>/speckit.tasks</code>
Analysis	<code>.specify/specs/<branch>/analyze.md</code>	<code>/speckit.analyze</code>
Design Doc	<code>~/.gstack/projects/<project>.md</code>	<code>/speckit.gstack-handoff</code>
Product Review	<code>specs/<feature>/product-review.md</code>	<code>/plan-ceo-review</code> output
Engineering Review	<code>specs/<feature>/engineering-review.md</code>	<code>/plan-eng-review</code> output
Code Review	<code>specs/<feature>/code-review.md</code>	<code>/review</code> output
Security Review	<code>specs/<feature>/security-review.md</code>	<code>/cso</code> output
QA Report	<code>specs/<feature>/qa-report.md</code>	<code>/qa</code> output
Release Notes	<code>specs/<feature>/release-notes.md</code>	<code>/ship</code> output
Retro	<code>specs/<feature>/retro.md</code>	<code>/retro</code> output

Phase Details

1. Define (30 min – 2 hrs)

```
bash

/speckit.constitution # Principles, standards, constraints, GStack review matrix
/speckit.specify      # Problem, user outcomes, acceptance criteria, test dimensi
/speckit.plan         # Architecture, data model, failure modes, test plan, review
/speckit.tasks        # Ordered units with dependencies, complexity, delivery gat
/speckit.analyze      # Cross-artifact consistency check (constitution conflicts
```

Spec template additions (via `gstack-ready` preset):

- Acceptance criteria table: ID, criterion, verification type, priority
- Test dimensions table: flow, type, browsers, data setup

Plan template additions:

- Architecture diagram (mermaid/ASCII)
- Edge cases & failure modes table
- Test plan: unit / integration / browser / security
- Review gate readiness checklist

Task template additions per task:

- Success criteria (from spec)
- Delivery gates: `[ ] code complete` `[ ] /review` `[ ] /qa` `[ ] /cso` `[ ] /ship`

2. Challenge (30 min - 1 hr)

```
bash

/speckit.gstack-handoff # Synthesize spec + plan + tasks → GStack design doc
/plan-ceo-review        # Is this the right thing to build? Scope decision.
/plan-eng-review        # Lock architecture, data flow, edge cases, test strategy
/plan-design-review     # UI changes only – 80-item audit, AI slop detection
```

Save review outputs to `specs/<feature>/product-review.md` and `engineering-review.md`.

Exit criterion: Eng Review Readiness Dashboard shows CLEARED.

3. Build (2-4 hrs per task)

```
bash
```

```
git checkout -b feature/<NNN>--<description>
```

```
# For each task:
```

```
# 1. Implement (use /speckit.implement or code directly)
```

```
# 2. Review
```

```
/review # Staff engineer persona, auto-fixes minor issues
```

```
# 3. QA
```

```
/qa # Real Chromium browser, tests spec acceptance criteria
```

```
# 4. Security (if task touches auth/payments/user data)
```

```
/cso # OWASP Top 10 + STRIDE
```

```
# 5. Ship
```

```
/ship # Sync, test, audit coverage, update docs, open PR
```

```
# 6. Mark delivery gates complete in tasks.md
```

Save findings: `code-review.md`, `qa-report.md`, `security-review.md`, `release-notes.md`.

4. Reflect (15 min weekly)

```
bash
```

```
/retro # Velocity, shipping streaks, test health, growth opportunit
```

Save to `specs/<feature>/retro.md`. Compare velocity against tasks.md estimates.

Installation

One-time setup

```
bash
```

```
# Spec-Kit
```

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v0.4.2
```

```
# GStack
```

```
git clone https://github.com/garrytan/gstack.git ~/.claude/skills/gstack
```

```
cd ~/.claude/skills/gstack && ./setup
```

Per-project setup

```
bash
```

```
# Initialize
specify init . --ai claude --here
specify preset add gstack-ready
specify extension add gstack-bridge

# Copy GStack into repo (makes project self-describing)
cp -Rf ~/.claude/skills/gstack .claude/skills/gstack
rm -rf .claude/skills/gstack/.git
cd .claude/skills/gstack && ./setup

# Create feature branch
git checkout -b feature/001-<description>
mkdir -p specs/001-<description>
```

CLAUDE.md Snippet

Drop this into your project root:

```
markdown
```

spec-kit + gstack

Spec-Kit for definition, GStack for delivery.

Workflow

1. Define: /speckit.constitution → /speckit.specify → /speckit.plan → /speckit.tasks
2. Bridge: /speckit.gstack-handoff
3. Challenge: /plan-ceo-review → /plan-eng-review
4. Build per task: implement → /review → /qa → /cso (if applicable) → /ship
5. Reflect: /retro

Artifacts

- Specs: .specify/specs/<branch>/
- Constitution: .specify/memory/constitution.md
- Review files: specs/<feature>/*.md

GStack skills

/office-hours, /plan-ceo-review, /plan-eng-review, /plan-design-review,
/design-consultation, /review, /ship, /land-and-deploy, /canary, /benchmark,
/browse, /qa, /qa-only, /design-review, /cso, /retro, /investigate,
/document-release, /codex, /autoplan, /careful, /freeze, /guard, /unfreeze

Bridge commands

/speckit.gstack-handoff, /speckit.gstack-review, /speckit.gstack-qa,
/speckit.gstack-ship, /speckit.gstack-sprint

GStack Integration Matrix

Reference during execution to engage the right specialist:

Role	Command	When	Gate
CEO	<code>/plan-ceo-review</code>	Before design lock	Scope decision made
Eng Manager	<code>/plan-eng-review</code>	Before coding	Architecture locked, test matrix defined
Designer	<code>/plan-design-review</code>	UI changes	Design approved, no AI slop
Staff Engineer	<code>/review</code>	Before <code>/ship</code>	No critical findings
QA Lead	<code>/qa</code>	After implementation	All spec flows pass in real browser
Security Officer	<code>/cso</code>	Auth/data/payments	OWASP + STRIDE pass
Release Engineer	<code>/ship</code>	Final gate	Tests pass, coverage target met, docs updated

File Structure

```
project-root/
├─ .specify/
│   ├── memory/constitution.md
│   ├── specs/feature/NNN-name/
│   │   ├── spec.md, plan.md, tasks.md, analyze.md
│   ├── presets/gstack-ready/
│   └─ extensions/gstack-bridge/
├─ .claude/
│   ├── skills/gstack/           # 28 specialist skills
│   └─ commands/                 # Slash commands (spec-kit + bridge)
├─ specs/
│   └─ NNN-feature-name/        # Persistent review artifacts
│       ├── product-review.md
│       ├── engineering-review.md
│       ├── code-review.md
│       ├── security-review.md
│       ├── qa-report.md
│       └─ release-notes.md
```

```
|      └─ retro.md
|  └─ CLAUDE.md
|  └─ src/
|  └─ tests/
```

Troubleshooting

Spec drift — After each `/ship`, update the relevant spec section. Run `/speckit.analyze` periodically.

Review churn — Front-load with `/plan-eng-review`. Lock architecture before coding. Most churn = undecided design, not bugs.

GStack can't find design doc — Run `/speckit.gstack-handoff` first. Confirm `~/.gstack/projects/<name>.md` exists.

Preset not applying — Check resolution: `specify preset resolve spec-template`. Re-run template generation for existing specs if preset was installed after `specify init`.

Key Metrics

Metric	Target
Spec-to-first-task shipped	Hours, not days
Review cycles per task	1-2 (not 3-5)
Test coverage	85%+ (100% on auth/payments)
Spec flow coverage by QA	100% of acceptance criteria
Post-ship bugs per sprint	0-1

The real metric: Confidence at ship time. By `/ship`, code has been specified, planned, challenged, reviewed, browser-tested, and security-audited.